

HYBRID MACHINE LEARNING FOR TABULAR DATA

DE BOISVILLIERS Bob, BONNELLY Christopher, ROBERT Calixte

July 2026

1 Introduction

Tabular data represents one of the most widespread forms of data in both professional and scientific settings today. Whether in finance, healthcare, marketing, or industry, a large share of the information used in practice is structured in tables, organized into rows (observations) and columns (features). Unlike images or text, this type of data is inherently heterogeneous, combining numerical, categorical, and sometimes temporal variables, which raises specific challenges for automated analysis.

Machine learning applied to tabular data aims to extract predictive or explanatory models from these structured datasets. While deep neural networks have revolutionized image and text processing, tree-based methods such as Random Forests and gradient boosting algorithms remain among the most effective and widely used approaches for this kind of data.

In this report, we provide a comprehensive overview of machine learning methods for tabular data. We begin by introducing the evaluation metrics commonly used to assess model performance, we then present the main families of algorithms applied to tabular data, ranging from classical approaches to tree-based methods including decision trees and gradient boosting, whose underlying principles we detail. We further discuss more recent deep learning architectures designed specifically for tabular data. Finally, we compare these methods empirically on one or several datasets, analyzing their respective strengths and weaknesses in terms of predictive performance, training efficiency, and interpretability, in order to identify the approaches best suited to concrete prediction problems.

Our aim is to explain simply but with enough details so that anyone can understand basics and advanced knowledge in Machine Learning to start a new discussion in a second time for what Quantum Computers can provide to Machine Learning in any task based on the performance of any algorithm or method.

2 What is Tabular Data?

Tabular data is the most common way information is organized in science, engineering, business, and industry. It consists of a table where each row represents an

observation (also called a sample, instance, or record), and each column represents a variable describing that observation, commonly referred to as a feature. An additional column may contain the target variable that the machine learning model aims to predict.

Unlike unstructured data such as images, audio, or text, tabular data has an explicit structure in which every feature has a predefined semantic meaning. Features may be numerical (e.g., age, salary, temperature), categorical (e.g., gender, country, product category), ordinal (e.g., education level), binary (e.g., yes/no), or temporal (e.g., timestamps). A single dataset frequently combines several of these feature types, making tabular learning inherently heterogeneous.

A tabular dataset can be formally represented by a matrix

$$X \in \mathbb{R}^{N \times D},$$

where N denotes the number of observations and D the number of features. In supervised learning, each observation is associated with a target value

$$y \in \mathcal{Y},$$

where $\mathcal{Y}$ is either a finite set of classes for classification tasks or a continuous domain for regression problems. The objective of machine learning is to learn a function

$$f : X \rightarrow y$$

that generalizes well to unseen observations.

Although this representation appears simple, tabular data presents several challenges that distinguish it from other data modalities:

- heterogeneous feature types (continuous, categorical, ordinal, boolean);
- missing values that must often be handled explicitly;
- highly imbalanced target distributions;
- varying feature scales requiring normalization for some algorithms;
- complex nonlinear interactions between variables;

- relatively small datasets compared with those used in computer vision or natural language processing.

These characteristics explain why algorithms that excel on images or text, such as deep convolutional or transformer-based neural networks, do not necessarily achieve the best performance on tabular data. Instead, tree-based methods such as Decision Trees, Random Forests, XGBoost, LightGBM, and CatBoost have become the state-of-the-art for many practical applications due to their ability to naturally handle heterogeneous features, missing values, and nonlinear feature interactions.

Typical applications of tabular machine learning include credit scoring, fraud detection, customer churn prediction, medical diagnosis, insurance risk assessment, recommendation systems, industrial quality control, and many scientific data analysis problems. Since the majority of structured data collected by companies and institutions is stored in relational databases or spreadsheets, tabular learning remains one of the most important branches of modern machine learning.

3 Metrics in Tabular Machine Learning

This section presents the main metrics used in machine learning on tabular data. They are grouped according to their nature: classification, regression, probabilistic, calibration, and complexity metrics. These metrics are widely used in practice to measure the effectiveness of an algorithm depending on its intended use case.

3.1 Classification Metrics

Classification metrics apply when the target variable is discrete, i.e., it takes a finite set of values (classes). They allow us to evaluate a model's ability to correctly predict a label or to estimate a probability associated with each class.

In tabular problems, these metrics are particularly important because datasets often exhibit class imbalance, heterogeneous distributions, or asymmetric error costs.

3.1.1 Accuracy

Accuracy measures the proportion of correct predictions among all observations.

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(\hat{y}_i = y_i) \quad (1)$$

It is simple to interpret but can be misleading when classes are imbalanced, since a naive model can achieve high accuracy by systematically predicting the majority class.

3.1.2 Precision

Precision measures the proportion of true positives among positive predictions.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (2)$$

where TP is the number of correct positive predictions and FP is the number of incorrect positive predictions.

Just as simple to interpret, it complements accuracy (see Section 3.1.1) within a given class and is particularly important when false positives are costly.

3.1.3 Recall

Recall measures the model's ability to detect actual positives.

$$\text{Recall} = \frac{TP}{TP + FN} \quad (3)$$

Recall quantifies the error rate within a given class and how well it is represented by the algorithm. It reveals whether certain classes are well or poorly captured by the model according to specific criteria, and is crucial when false negatives are critical.

3.1.4 F1-score

The F1-score combines precision and recall into a single metric.

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4)$$

It is particularly useful when classes are imbalanced and a trade-off between the two types of errors is desired.

3.1.5 Confusion Matrix

$$\begin{bmatrix} TP & FP \\ FN & TN \end{bmatrix} \quad (5)$$

The confusion matrix is the fundamental tool from which most classification metrics (accuracy, precision, recall, F1) are derived. It cross-tabulates the model's predictions against the actual values:

- **TP (True Positive):** the model predicts positive, and it is actually positive.
- **FP (False Positive):** the model predicts positive, but it is actually negative (Type I error).
- **FN (False Negative):** the model predicts negative, but it is actually positive (Type II error).
- **TN (True Negative):** the model predicts negative, and it is actually negative.

For a multi-class problem, the matrix generalizes to a $K \times K$ matrix (K being the number of classes), where each row represents the true class and each column the predicted class. It allows recurring confusions between certain classes to be visually identified, which an aggregate metric such as accuracy cannot show.

3.2 Probabilistic Metrics

Unlike classical classification metrics, which evaluate a final decision (0 or 1), probabilistic metrics assess the quality of the probability estimate p_i produced by the model before thresholding. They are essential when one seeks not only a correct predicted class but also a reliable confidence associated with that prediction (e.g., medical contexts, credit scoring).

3.2.1 Log Loss (Cross-Entropy)

$$\text{LogLoss} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)] \quad (6)$$

Log Loss heavily penalizes confident but incorrect predictions: if the model predicts $p_i = 0.99$ for an observation that is actually negative ($y_i = 0$), the penalty will be very large (the term $\log(1 - p_i)$ tends toward $-\infty$ as $p_i \rightarrow 1$). Conversely, a cautious prediction (p_i close to 0.5) is penalized less in case of error.

This metric is widely used to train models (it is the native loss function of logistic regression and many gradient boosting algorithms), as it is differentiable. The lower the Log Loss, the better the quality of the predicted probabilities. Unlike accuracy, it accounts for the degree of confidence, not just the predicted class.

3.2.2 ROC-AUC

$$\text{ROC-AUC} = P(s(x^+) > s(x^-)) \quad (7)$$

The ROC (Receiver Operating Characteristic) curve plots the true positive rate (TPR, or recall) against the false positive rate (FPR) across all possible decision thresholds. The AUC (Area Under the Curve) summarizes this curve into a single value between 0 and 1.

It can be interpreted as the probability that a randomly chosen positive example receives a higher score than a randomly chosen negative example. An AUC of 0.5 corresponds to a random model, while an AUC of 1 corresponds to perfect separation between classes.

Important limitation: ROC-AUC can give a misleadingly favorable impression of performance on strongly imbalanced datasets, since the FPR remains low even with a large number of false positives if the negative class is dominant. In such cases, PR-AUC (see next section) is more informative.

3.2.3 PR-AUC (Precision-Recall AUC)

$$\text{PR-AUC} = \int_0^1 \text{Precision}(r) d\text{Recall}(r) \quad (8)$$

The Precision-Recall curve plots precision as a function of recall across different decision thresholds. PR-AUC summarizes the area under this curve.

Unlike ROC-AUC, this metric focuses solely on the positive class and ignores true negatives, making it much more sensitive and relevant in cases of strong class imbalance (e.g., fraud detection, where the positive class represents less than 1% of observations). A high PR-AUC means the model maintains good precision even at high recall levels, which is harder to achieve than a good ROC-AUC in this context.

3.3 Regression Metrics

3.3.1 MAE

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i| \quad (9)$$

Mean Absolute Error.

3.3.2 MSE

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (10)$$

Heavily penalizes large errors.

3.3.3 RMSE

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2} \quad (11)$$

An interpretable version of MSE, expressed in the same unit as the target variable.

3.3.4 R^2 score

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2} \quad (12)$$

Proportion of variance explained by the model.

3.4 Calibration Metrics

A well-calibrated model is one whose predicted probabilities faithfully reflect the actual frequency of occurrence of the event. For example, among all observations for which the model predicts a probability of 0.7, about 70% should actually belong to the positive class. Calibration is independent of the model's discriminative ability (measured, for instance, by ROC-AUC): a model can classify examples very well while still being poorly calibrated.

3.4.1 Brier Score

$$\text{Brier} = \frac{1}{N} \sum_{i=1}^N (p_i - y_i)^2 \quad (13)$$

The Brier Score is the equivalent of a mean squared error (MSE) applied to probabilities rather than continuous values. It ranges between 0 (perfect probabilities) and 1 (completely wrong probabilities).

It combines both the model’s discriminative ability and its calibration: a model can achieve a good Brier Score only if it is both accurate in its class predictions and well calibrated in its probabilities. It is often used as a softer alternative to Log Loss, since it does not diverge to infinity in the case of a confident error.

3.4.2 Expected Calibration Error (ECE)

$$\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{N} |\text{acc}(B_m) - \text{conf}(B_m)| \quad (14)$$

ECE measures the average gap between the model’s confidence and its actual accuracy. Concretely, predictions are grouped into M bins according to their predicted probability (e.g., $[0, 0.1]$, $[0.1, 0.2]$, ..., $[0.9, 1]$). For each bin B_m , we compute:

- $\text{conf}(B_m)$: the average predicted confidence in that bin,
- $\text{acc}(B_m)$: the actual proportion of positive examples in that bin.

ECE is the weighted average of the absolute differences between these two quantities. An ECE close to 0 indicates a well-calibrated model. This metric is often visualized using a **reliability diagram**, which plots $\text{acc}(B_m)$ against $\text{conf}(B_m)$: the closer the points are to the diagonal, the better the calibration.

3.5 Robustness and Generalization Metrics

A performance metric computed on a single train/test split can be misleading, as it strongly depends on the randomness of that split. Robustness metrics aim to estimate not only a model’s average performance but also its stability across different data samplings, which is a key indicator of its generalization ability.

3.5.1 Cross-validation score

$$\text{CV} = \frac{1}{K} \sum_{k=1}^K s_k \quad (15)$$

K -fold cross-validation consists of splitting the dataset into K equally sized subsets. The model is trained K times, each time on $K-1$ folds and evaluated on the remaining fold. This yields K scores s_k (e.g.,

accuracy or RMSE, depending on the task), whose average provides a more reliable estimate of model performance than a single train/test split, since it uses the entire dataset for both training and validation.

3.5.2 Inter-fold variance

$$\text{Var}_{CV} = \frac{1}{K} \sum_{k=1}^K (s_k - \bar{s})^2 \quad (16)$$

Inter-fold variance complements the average cross-validation score by measuring the dispersion of the performances s_k obtained across each fold. A model may have a good average score but a high variance, which reflects strong sensitivity to the data split (instability) and therefore a risk of poor generalization to new data.

Low variance is a sign of robustness: the model produces similar performance regardless of the subset of data used for training and testing. This metric is particularly useful for comparing two models with similar average scores but very different behaviors in terms of stability.

3.6 Complexity Metrics

Beyond predictive performance, the choice of a model in a real-world context (production, hardware constraints, real-time requirements) strongly depends on its computational cost. These metrics allow the trade-off between performance and required resources to be evaluated, a criterion that is often decisive when comparing models with similar predictive performance.

3.6.1 Training time

$$T_{train} = \sum_{b=1}^B t_b \quad (17)$$

Training time corresponds to the sum of the times t_b required to train the model on each of the B batches (or iterations/trees, depending on the algorithm). This metric is particularly important for gradient boosting models, whose training time grows with the number of trees, or for neural networks, where it depends on the number of epochs. It determines the feasibility of frequent model retraining in production (e.g., daily retraining on new data).

3.6.2 Inference time

$$T_{infer} = \frac{1}{N} \sum_{i=1}^N t(x_i) \quad (18)$$

Inference time measures the average time required to produce a prediction for an observation x_i . This metric is critical in applications requiring real-time responses (on-the-fly fraud detection, recommendation systems), where even a highly accurate model may be discarded if

it is too slow at execution. Tree-based models (Random Forest, gradient boosting) generally have lower inference times than deep neural networks on tabular data.

3.6.3 Memory footprint

$$M_{peak} = \max_t M(t) \quad (19)$$

Memory footprint corresponds to the peak memory $M(t)$ used by the model over time, whether during training (storage of gradients, intermediate data) or at inference (loading the model into memory). This metric is decisive for deploying models on resource-constrained systems (edge computing, mobile applications) or when a large number of models must be served simultaneously.

3.7 Summary

In tabular machine learning, no single metric is sufficient to evaluate a model. Modern evaluation relies on a combination of:

- predictive performance (Accuracy, AUC, RMSE),
- probabilistic quality (LogLoss, Brier, ECE),
- statistical robustness (CV, variance),
- computational cost (training and inference time),
- stability and generalization.

4 Which models are actually used in tabular data

The needs of the practitioner determine which family of models is appropriate. Interpretability, training speed, inference latency, memory consumption, and dataset size all influence the choice. This section surveys the methods that are deployed in practice, from established baselines to recent deep learning architectures.

4.1 Interpretability-first: Logistic regression, shallow trees, SHAP and LIME

In regulated domains such as credit scoring, healthcare, and insurance, interpretability is often an important requirement. Practitioners use inherently interpretable models and pair them with post-hoc explanation tools.

Logistic regression (with L1 or L2 regularization) is commonly used for binary and multiclass classification. Its coefficients are directly interpretable as log-odds contributions. In Python, the standard implementation is `LogisticRegression` from `scikit-learn` [37]. In R, the `glm` function with `family="binomial"` is the standard entry point.

Decision trees with limited depth are also used for their visual interpretability. Each split corresponds to a simple rule, and the tree can be inspected directly. The `DecisionTreeClassifier` and `DecisionTreeRegressor` in `scikit-learn` are the typical implementations.

These models are often paired with SHAP (SHapley Additive exPlanations) [35] or LIME (Local Interpretable Model-agnostic Explanations) [36]. The `shap` and `lime` Python packages are the reference implementations of these methods. SHAP provides both local explanations and aggregated measures of global feature importance, while LIME is primarily designed to explain individual predictions.

4.2 Gradient boosting machines

Gradient boosting machines are the most frequently used methods for tabular data. They handle heterogeneous features, missing values, and nonlinear interactions effectively. Three implementations dominate.

4.2.1 XGBoost

XGBoost [31] is an established implementation that introduced built-in regularization, efficient handling of sparse data, and cache-aware block structures. It is used in credit risk modeling, retail demand prediction, and customer churn analysis. The Python package `xgboost` integrates with `scikit-learn` pipelines. Hyperparameter tuning is often performed with Optuna or Hyperopt.

4.2.2 LightGBM

LightGBM [32], developed by Microsoft, uses Gradient-based One-Side Sampling (GOSS) and Exclusive Feature Bundling (EFB). This reduces training time and memory usage. It is frequently chosen for large-scale datasets. The Python package `lightgbm` provides a `scikit-learn` compatible API.

4.2.3 CatBoost

CatBoost [33], developed by Yandex, processes categorical features natively using an ordered target encoding that avoids target leakage. It is used in manufacturing, predictive maintenance, and metallurgy. The Python package `catboost` also provides a `scikit-learn` compatible API.

4.3 Random forest

Random forests [34] remain a robust baseline. They are parallelizable, require minimal hyperparameter tuning, and are less prone to overfitting on small datasets compared to gradient boosting. The `RandomForestClassifier` and

`RandomForestRegressor` in `scikit-learn` are widely used.

4.4 AutoML

AutoML platforms automate preprocessing, feature engineering, model selection, and hyperparameter optimization. They are used to rapidly benchmark multiple algorithms.

AutoGluon [38], developed by Amazon, stacks multiple gradient boosting machines, neural networks, and KNN models. H2O.ai and DataRobot are enterprise platforms with both open-source and commercial offerings. These tools are used to establish baseline performance quickly.

4.5 Deep learning for tabular data

Recent architectures have been designed specifically for tabular data. While not yet the default choice for most industrial problems, they are gaining traction on large datasets and in settings where a single model must generalize across many tables.

4.5.1 TabNet

TabNet [39], introduced by Google Cloud AI, uses sequential attention to select features at each decision step. The attention masks provide interpretability. It is available in Python via the `pytorch-tabnet` package.

4.5.2 TabPFN

TabPFN (Tabular Prior-Data Fitted Network) [40] is a foundation model for tabular data developed by Prior Labs. It is pre-trained on hundreds of millions of synthetic datasets generated via structural causal models. It approximates Bayesian inference through in-context learning, enabling predictions on new datasets without per-dataset training.

TabPFN is ranked at the top of TabArena, a benchmark for tabular foundation models. The enterprise version supports up to 10 million rows. Case studies on the Prior Labs website include applications with Hitachi and Oxford Cancer Analytics. In 2026, SAP agreed to acquire Prior Labs, indicating industry interest in this approach. The model is open-source and available in Python via the `tabpfn` package. Its main selling point is its inference speed, cost and performance without ensembling or fine tuning.

4.5.3 TabFM

TabFM (Tabular Foundation Model), announced by Google Research on June 30, 2026, is a zero-shot foundation model. It uses alternating row and column attention and was trained on hundreds of millions of syn-

thetic datasets. It currently ranks first on TabArena. Google has announced plans to integrate TabFM directly into BigQuery, allowing zero-shot predictions via an “AI.PREDICT” SQL command. The model is open-source under Apache-2.0 and available on Hugging Face and GitHub. The Python package `tabfm` provides `TabFMClassifier` and `TabFMRegressor`. It is currently limited to at most 10 output classes under a non-commercial license. Its main drawback is its cost of inference, which will probably quickly get better in the next few months.

4.6 Summary

The choice of model depends on the use case. Table 1 summarizes the common recommendations.

The industry consensus, supported by benchmarks and case studies, is that gradient boosting machines remain the default choice for most tabular problems. Deep learning methods such as TabPFN and TabFM are the most significant recent development, but they are not yet mature enough to fully displace gradient boosting in production environments where cost, interpretability, and control are primary concerns.

5 Quantum bricks

5.1 Data encoding

Every quantum brick shares the same prerequisite: a classical row $x \in \mathbb{R}^d$ must first be turned into a quantum state. This step, called *data encoding* or *state preparation*, is not a mere technicality; it fixes the geometry of the space in which the quantum model operates and therefore conditions everything that follows. A poorly chosen encoding can make an otherwise powerful circuit useless.

Three strategies dominate in practice.

- **Basis encoding.** Each classical bit is mapped to a computational basis state, e.g. $x = 101 \mapsto |101\rangle$. It is intuitive but expensive: encoding d bits requires d qubits, and continuous features must first be discretized.
- **Angle encoding.** Each feature is written into the rotation angle of a qubit, typically $x_i \mapsto R_y(x_i) |0\rangle$. It uses one qubit per feature and maps naturally onto near-term hardware, which makes it the default choice for most tabular applications.
- **Amplitude encoding.** A vector of 2^n features is stored in the amplitudes of an n -qubit state,

$$|\psi_x\rangle = \frac{1}{\|x\|} \sum_{i=1}^{2^n} x_i |i\rangle. \quad (20)$$

Scenario	Recommended method
Maximum interpretability required (healthcare, credit scoring)	Logistic regression or shallow decision tree, optionally paired with SHAP or LIME for explanations
Strong general-purpose performance on tabular data	XGBoost, LightGBM, or CatBoost
Robust baseline with minimal tuning	Random forest
Automated benchmarking and model selection	AutoGluon or H2O AutoML
Many independent datasets with limited tuning budget	TabPFN or TabFM (zero-shot / in-context learning)
Large-scale datasets where deep learning resources are available	TabNet, FT-Transformer, or other tabular neural architectures when appropriate

Table 1: Recommended models for common tabular data scenarios.

It is exponentially compact (n qubits for 2^n features), but preparing such a state generally requires a circuit of depth exponential in n , which limits its use on current devices.

The encoding is what defines the quantum feature map $x \mapsto |\phi(x)\rangle$ used throughout this section. In particular, the expressiveness of the quantum kernels introduced below is a direct consequence of the encoding circuit chosen here.

5.2 Kernel methods

5.2.1 The classical kernel trick

A linear model such as a support vector machine (SVM) separates data with a hyperplane. Its power comes from the *kernel trick*: rather than mapping each point explicitly into a high-dimensional feature space through a map ϕ , the model only ever needs the inner products between mapped points,

$$k(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle. \quad (21)$$

The function k , the *kernel*, is computed directly without ever forming $\phi(x)$. On tabular data of moderate size, kernel SVMs remain competitive with tree-based models, especially when the number of samples is limited.

5.2.2 The quantum feature map

The idea is to let the feature map ϕ be the encoding of a classical point into a quantum state. A parameterized circuit U_ϕ prepares

$$x \mapsto |\phi(x)\rangle = U_\phi(x) |0\rangle^{\otimes n}. \quad (22)$$

The feature space is now the Hilbert space of n qubits, of dimension 2^n . The hope is to reach feature spaces that are classically intractable to represent, while the encoding circuit itself stays shallow.

5.2.3 Quantum kernel estimation

The quantum kernel is the squared overlap, or fidelity, between two encoded states:

$$k(x_i, x_j) = |\langle \phi(x_i) | \phi(x_j) \rangle|^2. \quad (23)$$

It is estimated on hardware by running the circuit $U_\phi^\dagger(x_j) U_\phi(x_i)$ on $|0\rangle^{\otimes n}$ and measuring how often the all-zero state $|0\rangle^{\otimes n}$ is recovered. Repeating this for every pair of training points fills the Gram matrix, which is then handed to a classical SVM. The learning itself is therefore classical; only the kernel evaluation is quantum.

5.2.4 QSVM and variational alternatives

Both families of methods share the same encoding circuit $U_\phi(x)$ from Equation (22), but use it in opposite ways. It helps to clearly distinguish two objects:

- $U_\phi(x)$: the **encoding** circuit, which realizes the map $x \mapsto |\phi(x)\rangle$ (angle encoding, amplitude encoding, etc.). It depends only on the data point x and contains *no* trainable parameters.
- $W(\theta)$: a **variational** circuit (or *ansatz*), whose parameters θ are adjustable.

QSVM. The quantum support vector machine trains nothing on the quantum side. The circuit U_ϕ is fixed once and for all, exactly as one would choose an RBF or polynomial kernel in the classical setting. It is used solely to estimate the kernel of Equation (23) and fill the Gram matrix, which is then handed to a classical SVM. The only learning is that of the SVM: the quantum computer is merely an *instrument for measuring similarity*.

VQC. The variational quantum classifier adds, after the encoding, a second parameterized circuit $W(\theta)$:

$$|0\rangle^{\otimes n} \xrightarrow{U_\phi(x)} |\phi(x)\rangle \xrightarrow{W(\theta)} |\psi(x, \theta)\rangle. \quad (24)$$

The final state is measured to produce a prediction $\hat{y}(x, \theta)$, a loss function is evaluated, and the parameters θ are adjusted by gradient descent (typically via the *parameter-shift rule*), just as one would train the weights of a neural network. The encoding U_ϕ stays frozen; it is $W(\theta)$, not U_ϕ , that is trained.

In summary, the QSVM keeps the quantum computer as a mere kernel-evaluation tool and delegates the decision to a classical model, whereas the VQC makes the quantum circuit the entire decision model, trained end-to-end.

5.2.5 Promise and limitations

A rigorous quantum advantage has so far only been shown on problems engineered specifically to exhibit it, not on generic tabular datasets. Three obstacles matter in practice. First, building the Gram matrix costs $O(N^2)$ circuit evaluations for N samples, which becomes prohibitive on large tables. Second, hardware noise corrupts the fidelity estimates of Equation (23). Third, and most fundamentally, *kernel concentration*: as the number of qubits n grows, quantum kernels tend to concentrate around a constant value, making distinct points nearly indistinguishable and the classifier no better than chance unless the encoding is carefully restricted. Quantum kernels are thus a promising but still fragile brick for real tabular problems.

5.3 Quantum Reservoir for temporal data

Tabular data is not always static. In many realistic settings, each row is not an independent observation but a snapshot taken at a given time: sensor readings from an industrial machine, daily transactions on a bank account, successive clinical measurements of a patient. In such cases the columns still look tabular, but the rows are correlated through time, and the quantity of interest often depends on the whole past trajectory rather than on the current row alone. Tree-based models, which dominate static tabular problems, have no built-in notion of memory: temporal dependencies must be injected by hand through lagged features or rolling statistics. Recurrent neural networks do carry an internal state, but training them by backpropagation through time is notoriously delicate and expensive [1]. Reservoir computing offers a third way, and it is this third way that admits a natural quantum generalization.

5.3.1 What is a reservoir in the classical sense?

A reservoir is a fixed, randomly initialized dynamical system that is never trained. The input signal is injected into it, the system reacts, and only a simple linear readout on top of its state is fitted to the target. Formally, given an input sequence $\{u_t\}$, the reservoir maintains a state $x_t \in \mathbb{R}^N$ that evolves according to

$$x_t = f(W_{\text{res}} x_{t-1} + W_{\text{in}} u_t), \quad (25)$$

where W_{in} and W_{res} are drawn at random and then frozen, and f is a nonlinearity (typically $\tanh$). The prediction is obtained by a linear map

$$\hat{y}_t = W_{\text{out}} x_t, \quad (26)$$

and W_{out} is the *only* trained object. Because Equation (26) is linear in the parameters, fitting the model amounts to a ridge regression **admitting a closed-form solution**, with no gradient descent, no back-propagation through time, and no vanishing-gradient pathology.

This idea appeared independently as Echo State Networks in machine learning [2] and as Liquid State Machines in computational neuroscience [3], and the two were later recognized as instances of a single paradigm [4, 5]. The intuition behind it is that of a stone dropped in a pond: the ripples that propagate on the surface encode, in a rich and nonlinearly mixed form, both the position and the timing of the impact. One does not need to train the water, only to learn how to read it.

For this to work, the reservoir must satisfy two properties. The **echo state property** requires that the influence of the initial condition vanishes asymptotically, so that x_t becomes a function of the input history alone; in practice this is controlled by rescaling the spectral radius of W_{res} [2, 6]. The **fading memory** property requires that the recent past weighs more than the distant past, which is exactly what is needed to approximate the causal, time-invariant filters that arise in real signals [7]. When these two conditions hold, the reservoir behaves as a high-dimensional nonlinear temporal feature map: it lifts a scalar or low-dimensional input stream into a space where a linear model is already expressive enough. Reservoirs sit at the edge of chaos, where memory and nonlinear mixing are simultaneously maximized [8], and their expressive power can be quantified rigorously through the information processing capacity, which is bounded by the number of linearly independent state variables [9].

That bound is precisely the pressure point. The quality of a reservoir grows with the dimension of its state space, but a classical reservoir of N neurons offers exactly N state variables, and simulating it costs $O(N^2)$ per time step.

5.3.2 Why move to quantum reservoir computing?

A quantum system of n qubits lives in a Hilbert space of dimension 2^n , and its density matrix has $4^n - 1$ real degrees of freedom. If the reservoir state is taken to be the quantum state itself, then the number of available state variables grows *exponentially* with the hardware size, rather than linearly [10]. Ten qubits already provide a state space that no classical reservoir of ten units could approach. This is the observation that founded quantum reservoir computing (QRC): Fujii and Nakajima showed that the natural Hamiltonian dynamics of a disordered spin network, left entirely untrained, already produces the nonlinear mixing and the fading memory that temporal learning requires [10].

The argument is stronger than a mere counting of dimensions, and rests on three points.

First, QRC is remarkably well suited to near-term hardware. The central difficulty of variational quantum algorithms is training: gradients must be estimated on the device, and they vanish exponentially with system size in the barren plateau phenomenon [11]. QRC sidesteps this entirely, because there is nothing to train inside the quantum device. The circuit is fixed, the optimization is a classical linear regression, and the noisy, imperfectly characterized dynamics of a NISQ processor [12] becomes a feature rather than a defect: disorder is exactly what a good reservoir needs [20].

Second, the natural decoherence of an open quantum system plays the role of the echo state property. In a closed unitary system, information about the initial condition never dissipates and the echo state property fails; dissipation, whether from the environment or from repeated measurements, erases the initial state and installs precisely the fading memory required [18, 24]. The imperfection of the hardware supplies for free a property that classical reservoirs must obtain by careful spectral tuning.

Third, the operating regime matters as much as the size. The memory capacity of a quantum reservoir depends critically on where the system sits with respect to the ergodic-to-localized transition, and it is maximized near the transition, the quantum analogue of the classical edge of chaos [19]. QRC has been demonstrated experimentally on NMR platforms [13] and analyzed in photonic and superconducting settings [14], and it should be said that the field is honest about its limits: recent work shows that the exponential state space is not automatically usable, since accessing it requires an exponential number of measurement shots unless the design is chosen with care [22].

5.3.3 The QRC pipeline

Figure 1 summarizes the full path from a classical temporal table to a classical prediction. Each stage is described below.

Classical data $\rightarrow$ feature map. The starting point is a multivariate time series $\{u_t\}_{t=1}^T$, that is, a table whose rows are ordered in time. As in any tabular pipeline, each input is first normalized, typically to $[0, 1]$ or $[-1, 1]$, because the encoding stage interprets the numerical value as an angle or an amplitude and is therefore not scale-invariant. The feature map is the classical-to-quantum interface: it associates to each input u_t a unitary $V(u_t)$ that will write the data into the device. The standard choice is angle encoding, $V(u_t) = \bigotimes_j R_Y(\pi u_t^{(j)})$, which consumes one qubit per feature and one circuit layer, but amplitude encoding and the entangling encodings of Havlíček *et al.* are also used [15]. This step is the exact analogue of W_{in} in Equation (25), and it is not innocuous: the choice of feature map fixes the kernel that the model implicitly computes, and therefore the class of functions it can represent [16, 17]. This is the point of contact with the kernel methods of the previous subsection.

Feature map $\rightarrow$ quantum data. Applying $V(u_t)$ produces the quantum state that encodes the current input. Two protocols coexist. In the *restart* protocol, the device is reinitialized at every step and the whole input history is re-injected, which is faithful but costs $O(T)$ circuit depth. In the *online* protocol, only a subset of qubits, the input qubits, is reset and re-encoded, while the remaining qubits retain the state produced at step $t - 1$ [10]:

$$\rho_t^{\text{in}} = V(u_t) [|\psi(u_t)\rangle\langle\psi(u_t)| \otimes \text{Tr}_{\text{in}}(\rho_{t-1})] V^\dagger(u_t). \quad (27)$$

The partial trace over the input subsystem is what makes the map non-unitary, and hence what makes the memory fade: without it, the echo state property would not hold [18].

Quantum data $\rightarrow$ QRC. The reservoir proper is the evolution of the encoded state under a fixed, untrained dynamics. In the Hamiltonian formulation one lets the system evolve for a time Δt under a disordered transverse-field Ising Hamiltonian,

$$H = \sum_{i < j} J_{ij} \sigma_i^x \sigma_j^x + \sum_i h_i \sigma_i^z, \quad (28)$$

with couplings J_{ij} drawn once at random and then frozen, exactly as W_{res} is in the classical case [10]. In the gate-based formulation, the same role is played by a fixed random circuit [21]. The state update reads $\rho_t = \mathcal{E}(\rho_t^{\text{in}})$, with $\mathcal{E}(\cdot) = e^{-iH\Delta t}(\cdot)e^{iH\Delta t}$ in the closed case, or a general CPTP map in the presence of noise. Two hyperparameters govern the whole regime: the evolution time Δt , which sets the trade-off between memory and nonlinearity, and the disorder strength, which places the system on one side or the other of the

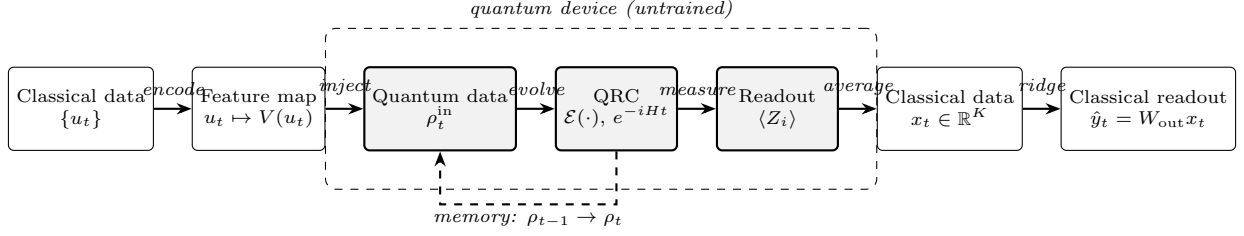


Figure 1: The quantum reservoir computing pipeline for temporal data. Everything inside the dashed box is fixed: no parameter of the quantum device is optimized. The only trained component is the final linear readout W_{out} , obtained in closed form by ridge regression. The dashed feedback arrow denotes the reservoir’s internal memory, the state ρ_{t-1} surviving into step t .

ergodic-to-localized transition [19]. These, and not the couplings themselves, are what one actually tunes.

QRC $\rightarrow$ readout $\rightarrow$ classical data. The quantum state is not directly accessible: it must be measured. One estimates a set of local observables, most commonly the single-qubit magnetizations $x_t^{(i)} = \text{Tr}(\rho_t \sigma_i^z)$, possibly enriched with two-qubit correlators $\text{Tr}(\rho_t \sigma_i^z \sigma_j^z)$, which yields the classical feature vector $x_t \in \mathbb{R}^K$. Two remarks are in order. First, each expectation value requires many repetitions of the circuit, and the shot noise scales as $O(1/\sqrt{S})$ for S shots; this measurement cost is the true bottleneck of QRC and the reason why the 4^n degrees of freedom are not all usable in practice [22]. Second, measurement is destructive, which conflicts with the online protocol; the temporal multiplexing of Fujii and Nakajima [10], and more recently the repeated-measurement schemes of [23], are designed to extract several virtual nodes per time step and thereby amortize this cost.

Classical data $\rightarrow$ classical readout. The final stage is the same as in the classical case, and it is deliberately trivial: a ridge regression of the target on the measured features,

$$W_{\text{out}} = \arg \min_W \sum_t \|y_t - Wx_t\|^2 + \lambda \|W\|^2, \quad (29)$$

solved in closed form by $W_{\text{out}} = YX^\top (XX^\top + \lambda I)^{-1}$. The regularization λ is not optional here: the measured features are correlated and shot-noisy, so $XX^\top$ is ill-conditioned, and λ is the parameter that keeps the readout from fitting the sampling noise. Everything upstream of Equation (29) is untrained, which means the entire training procedure is a single linear solve, executed classically, on features that the quantum device has produced.

5.3.4 Relevance for temporal tabular data

For the tabular problems that concern us, QRC is best understood not as a competitor to gradient boost-

ing but as an automatic feature extractor for temporal rows. Rather than hand-crafting lags and rolling means, one lets a fixed dynamical system generate a rich nonlinear temporal embedding x_t , and one is free to plug any classical model on top of it, including the tree-based methods discussed earlier. Its benefits and its costs are clearly delineated: no gradient training, robustness to hardware noise, and an exponentially large state space on the one hand; a measurement cost that grows with the number of observables, a limited number of available qubits, and a device access latency that makes it uncompetitive for the low-dimensional streams on which classical reservoirs already perform excellently on the other [20, 22]. The regime where QRC could genuinely pay off is that of highly multivariate temporal signals with long memory, where the classical reservoir would need a state space too large to simulate. This is a hypothesis that current hardware does not yet allow us to test at scale.

5.4 Feature Selection via Quantum Annealing

5.4.1 The classical starting point: mRMR

Real-world tabular datasets often contain far more features than are useful. Beyond a certain point, adding features does not improve a model but instead introduces redundancy, noise, and overfitting risk. Feature selection addresses this by choosing, among the D available features, the subset that is simultaneously the most *relevant* to the target and the least *redundant* internally.

The classical reference method for this trade-off is **mRMR** (*minimum Redundancy Maximum Relevance*) [25], which selects a subset S maximizing

$$\text{mRMR}(S) = \underbrace{\frac{1}{|S|} \sum_{i \in S} I(x_i, y)}_{\text{relevance}} - \underbrace{\frac{1}{|S|^2} \sum_{i, j \in S} I(x_i, x_j)}_{\text{redundancy}}, \quad (30)$$

where $I(\cdot, \cdot)$ denotes mutual information (or, in practice, a simpler proxy such as the Pearson correlation). Consider two features frequently found together in real-estate datasets: the *surface area* of an apartment and its *current listed price*. Both are individually informative about the future selling price, but they are also strongly correlated with each other, since larger apartments tend to be more expensive. mRMR formalizes exactly the arbitration a practitioner would do by hand: keep both features only if their shared information is small relative to what each contributes on its own; otherwise, drop the one that is least incrementally informative once the other is already included.

mRMR is a greedy, well-established heuristic, but it is precisely that: a heuristic. Solving it exactly means testing all 2^D subsets of features, and for any but the smallest D this is intractable. This is where a reformulation as a discrete optimization problem becomes useful, since it opens the door to solvers that were never designed for feature selection at all, quantum annealers among them.

5.4.2 From mRMR to a QUBO

Feature selection is naturally binary: each feature is either kept or discarded. Let $x_i \in \{0, 1\}$ indicate whether feature i is selected. The mRMR objective (Equation 30) can be rewritten, up to the normalization constants, as a **Quadratic Unconstrained Binary Optimization** (QUBO) problem:

$$\min_{x \in \{0, 1\}^D} x^\top Q x = \sum_{i=1}^D Q_{ii} x_i + \sum_{i < j} Q_{ij} x_i x_j, \quad (31)$$

where the matrix Q is built directly from data statistics:

- $Q_{ii} = -\text{relevance}(x_i, y)$: a large negative diagonal term rewards keeping a feature that is individually predictive of the target;
- $Q_{ij} = +\text{redundancy}(x_i, x_j)$ for $i \neq j$: a large positive off-diagonal term penalizes keeping two features that carry overlapping information.

Returning to the apartment example, with x_{surf} and x_{price} denoting the two binary selection variables: if surface area and current price are correlated at, say, 0.9, the corresponding Q_{ij} term is large and positive, making the joint assignment $x_{\text{surf}} = x_{\text{price}} = 1$ costly. The optimizer is then pushed to zero out whichever of the two has the smaller individual relevance term Q_{ii} — typically the surface area, since the current price already implicitly reflects the surface through the market. Conversely, if their correlation were only 0.4, the redundancy penalty would be too small to justify discarding either feature, and the optimal solution would keep

both. This is exactly the mRMR trade-off of Equation (30), only rewritten in a form that a very different family of solvers can attack directly.

5.4.3 Solving the QUBO with a quantum annealer

The QUBO form of Equation (31) is, up to the change of variables $x_i = (s_i + 1)/2$ with $s_i \in \{-1, +1\}$, mathematically identical to finding the ground state of an Ising Hamiltonian,

$$H_{\text{Ising}} = \sum_{i < j} J_{ij} s_i s_j + \sum_i h_i s_i, \quad (32)$$

with couplings J_{ij} and local fields h_i obtained by direct substitution from Q . This is precisely the physical quantity that quantum annealing hardware, such as D-Wave processors, is built to minimize: the qubits are initialized in an easily prepared ground state and slowly evolved, via the adiabatic theorem, toward the ground state of H_{Ising} [26, 27]. If the evolution is slow enough relative to the spectral gap, the system remains in, or close to, its instantaneous ground state throughout, and the final measurement of $\{s_i\}$ yields a candidate optimal feature subset. Along the way, quantum tunneling allows the system to cross narrow energy barriers between local minima that a classical simulated annealer would have to climb over thermally, which is the source of the (task-dependent, and not universally realized) speed-up expected from this approach [28].

Two things are worth stressing about this pipeline compared to the QRC pipeline described earlier in this report. First, there is no notion of a temporal sequence here: the annealer solves a single, static optimization problem, so the feature-selection use case targets the same tabular data as gradient boosting and random forests, rather than the temporal tables addressed by QRC. Second, and just as for QRC, there is strictly speaking no “training” of the quantum device itself: the couplings J_{ij} are computed once from the dataset statistics and handed directly to the hardware, which performs a single annealing run.

5.4.4 The gate-based alternative: QAOA

On gate-based hardware, the same QUBO can instead be tackled with the **Quantum Approximate Optimization Algorithm** (QAOA) [29]. A cost Hamiltonian H_C , built from Q exactly as in Equation (32), is alternated with a mixing Hamiltonian $H_M = \sum_i \sigma_i^x$ across p layers, producing a parametrized state

$$|\psi(\gamma, \beta)\rangle = \prod_{k=1}^p e^{-i\beta_k H_M} e^{-i\gamma_k H_C} |+\rangle^{\otimes D}, \quad (33)$$

whose parameters (γ, β) are tuned by a classical optimizer to minimize $\langle \psi | H_C | \psi \rangle$, in the same variational

loop used by the quantum neural networks discussed elsewhere in the quantum machine learning literature. This is the crucial contrast with both quantum annealing and the QRC pipeline of the previous subsection: QAOA *does* require training, and its gradients are estimated on real, noisy hardware. As such, it inherits the barren plateau phenomenon already mentioned in the context of variational circuits [11], whereby gradients vanish exponentially as the number of features D grows, making the classical optimization loop increasingly difficult to converge in practice.

5.4.5 Limitations

As with QRC, the practical relevance of this pipeline should not be overstated. Three limitations deserve mention. First, current annealers offer a large but fixed number of physical qubits with limited connectivity, so a fully connected QUBO such as Equation (31) generally requires *minor embedding*, mapping each logical variable onto a chain of physical qubits; this consumes hardware resources and degrades solution quality as D grows [30]. Second, the QUBO form is strictly quadratic, so any higher-order interaction between three or more features (a plausible scenario in tabular data, where the joint effect of surface area, number of rooms, and floor level may not decompose pairwise) must be reduced to pairwise terms through auxiliary variables, further inflating the problem size. Third, and as for QRC, no controlled, large-scale empirical study currently demonstrates a clear advantage of quantum annealing or QAOA over classical feature-selection heuristics such as mRMR or LASSO on realistic tabular datasets; the case for this approach remains, at present, structural rather than demonstrated.

References

- [1] Y. Bengio, P. Simard, and P. Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166, 1994.
- [2] H. Jaeger. The “echo state” approach to analysing and training recurrent neural networks. Technical Report GMD Report 148, German National Research Center for Information Technology, 2001.
- [3] W. Maass, T. Natschlager, and H. Markram. Real-time computing without stable states: a new framework for neural computation based on perturbations. *Neural Computation*, 14(11):2531–2560, 2002.
- [4] D. Verstraeten, B. Schrauwen, M. D’Haene, and D. Stroobandt. An experimental unification of reservoir computing methods. *Neural Networks*, 20(3):391–403, 2007.
- [5] M. Lukoševičius and H. Jaeger. Reservoir computing approaches to recurrent neural network training. *Computer Science Review*, 3(3):127–149, 2009.
- [6] I. B. Yildiz, H. Jaeger, and S. J. Kiebel. Re-visiting the echo state property. *Neural Networks*, 35:1–9, 2012.
- [7] S. Boyd and L. O. Chua. Fading memory and the problem of approximating nonlinear operators with Volterra series. *IEEE Transactions on Circuits and Systems*, 32(11):1150–1161, 1985.
- [8] N. Bertschinger and T. Natschlager. Real-time computation at the edge of chaos in recurrent neural networks. *Neural Computation*, 16(7):1413–1436, 2004.
- [9] J. Dambre, D. Verstraeten, B. Schrauwen, and S. Massar. Information processing capacity of dynamical systems. *Scientific Reports*, 2:514, 2012.
- [10] K. Fujii and K. Nakajima. Harnessing disordered-ensemble quantum dynamics for machine learning. *Physical Review Applied*, 8(2):024030, 2017.
- [11] J. R. McClean, S. Boixo, V. N. Smelyanskiy, R. Babbush, and H. Neven. Barren plateaus in quantum neural network training landscapes. *Nature Communications*, 9:4812, 2018.
- [12] J. Preskill. Quantum computing in the NISQ era and beyond. *Quantum*, 2:79, 2018.
- [13] M. Negoro, K. Mitarai, K. Fujii, K. Nakajima, and M. Kitagawa. Machine learning with controllable quantum dynamics of a nuclear spin ensemble in a solid. *arXiv:1806.10910*, 2018.
- [14] S. Ghosh, A. Opala, M. Matuszewski, T. Paterek, and T. C. H. Liew. Quantum reservoir processing. *npj Quantum Information*, 5:35, 2019.
- [15] V. Havlíček, A. D. Córcoles, K. Temme, A. W. Harrow, A. Kandala, J. M. Chow, and J. M. Gambetta. Supervised learning with quantum-enhanced feature spaces. *Nature*, 567:209–212, 2019.
- [16] M. Schuld and N. Killoran. Quantum machine learning in feature Hilbert spaces. *Physical Review Letters*, 122(4):040504, 2019.
- [17] M. Schuld, R. Sweke, and J. J. Meyer. Effect of data encoding on the expressive power of variational quantum machine learning models. *Physical Review A*, 103(3):032430, 2021.
- [18] J. Chen, H. I. Nurdin, and N. Yamamoto. Temporal information processing on noisy quantum computers. *Physical Review Applied*, 14(2):024065, 2020.

- [19] R. Martínez-Peña, G. L. Giorgi, J. Nokkala, M. C. Soriano, and R. Zambrini. Dynamical phase transitions in quantum reservoir computing. *Physical Review Letters*, 127(10):100502, 2021.
- [20] P. Mujal, R. Martínez-Peña, J. Nokkala, J. García-Bení, G. L. Giorgi, M. C. Soriano, and R. Zambrini. Opportunities in quantum reservoir computing and extreme learning machines. *Advanced Quantum Technologies*, 4(8):2100027, 2021.
- [21] Y. Suzuki, Q. Gao, K. C. Pradel, K. Yasuoka, and N. Yamamoto. Natural quantum reservoir computing for temporal information processing. *Scientific Reports*, 12:1353, 2022.
- [22] L. Innocenti, S. Lorenzo, I. Palmisano, A. Ferraro, M. Paternostro, and G. M. Palma. Potential and limitations of quantum extreme learning machines. *Communications Physics*, 6:118, 2023.
- [23] P. Mujal, R. Martínez-Peña, G. L. Giorgi, M. C. Soriano, and R. Zambrini. Time-series quantum reservoir computing with weak and projective measurements. *npj Quantum Information*, 9:16, 2023.
- [24] A. Sannia, R. Martínez-Peña, M. C. Soriano, G. L. Giorgi, and R. Zambrini. Dissipation as a resource for quantum reservoir computing. *Quantum*, 8:1291, 2024.
- [25] H. Peng, F. Long, and C. Ding. Feature selection based on mutual information: criteria of max-dependency, max-relevance, and min-redundancy. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 27(8):1226–1238, 2005.
- [26] T. Kadowaki and H. Nishimori. Quantum annealing in the transverse Ising model. *Physical Review E*, 58(5):5355–5363, 1998.
- [27] E. Farhi, J. Goldstone, S. Gutmann, and M. Sipser. Quantum computation by adiabatic evolution. *arXiv:quant-ph/0001106*, 2000.
- [28] V. S. Denchev, S. Boixo, S. V. Isakov, N. Ding, R. Babbush, V. Smelyanskiy, J. Martinis, and H. Neven. What is the computational value of finite-range tunneling? *Physical Review X*, 6(3):031015, 2016.
- [29] E. Farhi, J. Goldstone, and S. Gutmann. A quantum approximate optimization algorithm. *arXiv:1411.4028*, 2014.
- [30] J. Cai, W. G. Macready, and A. Roy. A practical heuristic for finding graph minors. *arXiv:1406.2741*, 2014.
- [31] T. Chen and C. Guestrin. XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.
- [32] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T. Liu. LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 2017.
- [33] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin. CatBoost: unbiased boosting with categorical features. *Advances in Neural Information Processing Systems (NeurIPS)*, 31, 2018.
- [34] L. Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [35] S. M. Lundberg and S. Lee. A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 2017.
- [36] M. T. Ribeiro, S. Singh, and C. Guestrin. "Why should I trust you?": Explaining the predictions of any classifier. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1135–1144, 2016.
- [37] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [38] N. Erickson, J. Mueller, A. Shirkov, H. Zhang, P. Larroy, M. Li, and A. Smola. AutoGluon-Tabular: Robust and accurate AutoML for structured data. *arXiv preprint arXiv:2003.06505*, 2020.
- [39] S. O. Arik and T. Pfister. TabNet: Attentive interpretable tabular learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(8):6679–6687, 2021.
- [40] N. Hollmann, S. Müller, K. Eggenberger, and F. Hutter. TabPFN: A transformer that solves small tabular classification problems in a second. *Nature Machine Intelligence*, 2023.
- [41] Google Research. TabFM: Zero-shot tabular foundation models. *arXiv preprint*, 2026.